

SCR2043 OPERATING SYSTEMS

Name : MOHAMED ALIF FATHI BIN
ABDUL LATIF
Student ID : A23CS0112
Section : 02

Marks

This lab assessment is designed to test your understanding and skills on some basic concepts and tools related to process monitoring and management in operating system. Please follow the instructions carefully and submit your answers in this word document and rename the file as **os-lab-assessment02-studentname-matricno.docx**.

Essential Steps Before Starting Lab Assessment 2:

1. Download necessary source codes:

Use the `wget` command to retrieve the following source code files to your Linux (or WSL or MacOS) environment:

```
wget -O mainprocess.c https://rebrand.ly/mainprocess_c
wget -O subprocess1.c https://rebrand.ly/subprocess1_c
wget -O subprocess2.c https://rebrand.ly/subprocess2_c
```

2. Compile the source files:

Use the `gcc` compiler to create executable files from the source code.

```
gcc mainprocess.c -o mainprocess
gcc subprocess1.c -o subprocess1
gcc subprocess2.c -o subprocess2
```

3. Execute the dummy processes:

Run all the dummy processes

```
./mainprocess &
```

Press **enter** two times.

4. The dummy processes are running for 2 hours. If you took longer than 2 hours on questions 1-9, please restart the main process with `./mainprocess &`.

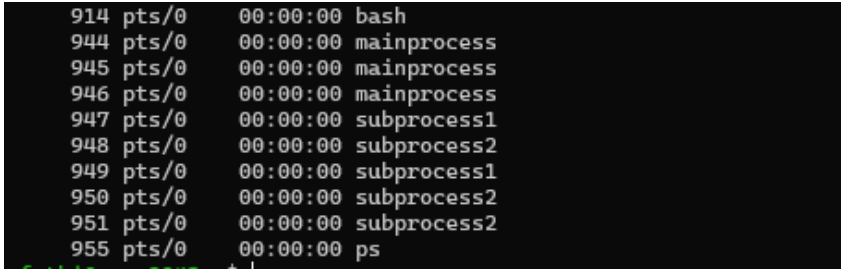
Lab Assessment 2 : Linux Process Monitoring and Management

Instructions:

1. Carefully execute each command as instructed in the questions.
2. Write down the exact command used for each task.
3. Capture a screenshot of the command's output.

Question 1

Use the `ps` command with the appropriate option to display a complete list of all running processes within the Linux operating system.

Command	
<code>ps -e</code>	
Output	
	

Question 2

Employ the `ps` command with necessary options to unveil comprehensive details about each running process.

Command												
ps aux												
Output												
fathi	914	0.0	0.5	5644	4992	pts/0	Ss	14:03	0:00	-bash		
fathi	944	0.0	0.1	2680	1536	pts/0	S	14:05	0:00	./mainprocess		
fathi	945	0.0	0.0	2680	128	pts/0	S	14:05	0:00	./mainprocess		
fathi	946	0.0	0.0	2680	128	pts/0	S	14:05	0:00	./mainprocess		
fathi	947	0.0	0.1	2680	1536	pts/0	S	14:05	0:00	./subprocess1		
fathi	948	0.0	0.1	2680	1536	pts/0	S	14:05	0:00	./subprocess2		
fathi	949	0.0	0.1	2680	1536	pts/0	S	14:05	0:00	./subprocess1		
fathi	950	0.0	0.1	2680	1536	pts/0	S	14:05	0:00	./subprocess2		
fathi	951	0.0	0.1	2680	1536	pts/0	S	14:05	0:00	./subprocess2		
root	956	0.0	0.0	0	0	?	I	14:07	0:00	[kworker/u6:0-events_power_efficient]		
root	957	0.0	0.0	0	0	?	I	14:09	0:00	[kworker/u5:2-events_unbound]		
root	960	0.0	0.0	0	0	?	I	14:09	0:00	[kworker/0:1]		
fathi	966	0.0	0.4	7892	4896	pts/0	R+	14:15	0:00	ps aux		

Question 3

Use the `ps` command with some tools to only list processes named "subprocess" and show some info about them.

Command												
ps aux grep subprocess												
Output												
fathi@secre2043:~\$	ps	aux		grep	subprocess							
fathi	947	0.0	0.1	2680	1536	pts/0	S	14:05	0:00	./subprocess1		
fathi	948	0.0	0.1	2680	1536	pts/0	S	14:05	0:00	./subprocess2		
fathi	949	0.0	0.1	2680	1536	pts/0	S	14:05	0:00	./subprocess1		
fathi	950	0.0	0.1	2680	1536	pts/0	S	14:05	0:00	./subprocess2		
fathi	951	0.0	0.1	2680	1536	pts/0	S	14:05	0:00	./subprocess2		
fathi	975	0.0	0.1	3528	1792	pts/0	S+	14:18	0:00	grep --color=auto subprocess		

Question 4

Execute the `ps` command, specifying options that reveal only the following columns:

- Process ID (`pid`)
- Owner of the process (`user`)
- CPU percentage (`pcpu`)
- Memory percentage (`pmem`)
- Command (`cmd`)

Command				
<code>ps -eo pid,user,pcpu,pmem,cmd</code>				
Output				
914	fathi	0.0	0.5	-bash
944	fathi	0.0	0.1	./mainprocess
945	fathi	0.0	0.0	./mainprocess
946	fathi	0.0	0.0	./mainprocess
947	fathi	0.0	0.1	./subprocess1
948	fathi	0.0	0.1	./subprocess2
949	fathi	0.0	0.1	./subprocess1
950	fathi	0.0	0.1	./subprocess2
951	fathi	0.0	0.1	./subprocess2
956	root	0.0	0.0	[kworker/u6:0-events_power_efficient]
957	root	0.0	0.0	[kworker/u5:2-events_power_efficient]
967	root	0.1	0.0	[kworker/0:2-events]
976	root	0.0	0.0	[kworker/u5:3-events_unbound]
979	root	0.0	0.0	[kworker/1:1]
980	root	0.0	0.0	[kworker/1:3]
983	fathi	0.0	0.4	ps -eo pid,user,pcpu,pmem,cmd

Question 5

Building on the ps command used in Question 4, can you add an option to sort the listed processes by their memory usage (pmem)?

Command
<code>ps -eo pid,user,pcpu,pmem,cmd --sort=-pmem</code>
Output
<pre>fathi@secur2043:~\$ ps -eo pid,user,pcpu,pmem,cmd --sort=-pmem PID USER %CPU %MEM CMD 358 root 0.0 2.7 /sbin/multipathd -d -s 758 root 0.0 2.3 /usr/bin/python3 /usr/share/unattended-upgrades/unattended-upgrade-shutdown --wait-for-signal 669 root 0.0 1.3 /usr/libexec/udisks2/udisksd 1 root 0.0 1.3 /sbin/init 535 systemd+ 0.0 1.3 /usr/lib/systemd/systemd-resolved 766 root 0.0 1.3 /usr/sbin/ModemManager 383 root 0.0 1.2 /usr/lib/systemd/systemd-journald 519 systemd+ 0.0 0.9 /usr/lib/systemd/systemd-networkd 666 root 0.0 0.8 /usr/lib/systemd/systemd-logind 918 root 0.0 0.8 sshd: /usr/sbin/sshd -D [listener] 0 of 10-100 startups 543 systemd+ 0.0 0.7 /usr/lib/systemd/systemd-timesyncd 651 polkitd 0.0 0.7 /usr/lib/polkit-1/polkitd --no-debug 368 root 0.0 0.7 /usr/lib/systemd/systemd-udevd 913 fathi 0.0 0.7 sshd: fathi@pts/0 744 syslog 0.0 0.6 /usr/sbin/rsyslogd -n -iNONE 645 message+ 0.0 0.5 @dbus-daemon --system --address=systemd: --nofork --nopidfile --systemd-activation --syslog-only 911 root 0.0 0.5 sshd: fathi [priv] 914 fathi 0.0 0.5 -bash 984 fathi 0.0 0.4 ps -eo pid,user,pcpu,pmem,cmd --sort=-pmem 836 root 0.0 0.2 /usr/sbin/cron -f -P 844 root 0.0 0.1 /sbin/agetty -o -p -- \u --noclear - linux 944 fathi 0.0 0.1 ./mainprocess 947 fathi 0.0 0.1 ./subprocess1 948 fathi 0.0 0.1 ./subprocess2 949 fathi 0.0 0.1 ./subprocess1 950 fathi 0.0 0.1 ./subprocess2 951 fathi 0.0 0.1 ./subprocess2 945 fathi 0.0 0.0 ./mainprocess 946 fathi 0.0 0.0 ./mainprocess</pre>

Question 6

Construct a command using ps, suitable options, and any additional tools to visualize the hierarchical structure (tree-like) of the following processes:

- "mainprocess"
- "subprocess1"
- "subprocess2"

Command
<code>ps -ef --forest grep -E "mainprocess subprocess1 subprocess2"</code>
Output
<pre>fathi@secur2043:~\$ ps -ef --forest grep -E "mainprocess subprocess1 subprocess2" fathi 944 0 14:05 pts/0 00:00:00 _ ./mainprocess fathi 945 0 14:05 pts/0 00:00:00 _ _ ./mainprocess fathi 947 0 14:05 pts/0 00:00:00 _ _ _ ./subprocess1 fathi 949 0 14:05 pts/0 00:00:00 _ _ _ ./subprocess1 fathi 946 0 14:05 pts/0 00:00:00 _ _ _ ./mainprocess fathi 948 0 14:05 pts/0 00:00:00 _ _ _ ./subprocess2 fathi 950 0 14:05 pts/0 00:00:00 _ _ _ ./subprocess2 fathi 951 0 14:05 pts/0 00:00:00 _ _ _ ./subprocess2 fathi 1024 0 14:40 pts/0 00:00:00 _ grep --color=auto -E mainprocess subprocess1 subprocess2</pre>

Question 7

Use `pstree` command with option that show the number of threads to each process.

Command

```
pstree -p -t
```

Output

```
ath1@secr2003:~$ pstree -p -t
systemd(1)─┬─ModemManager(766)─┬─{gdbus}(792)
│                               │   └─{gmain}(787)
│                               └─{pool-spawner}(789)
│
│   └─agetty(844)
│   └─cron(836)
│   └─dbus-daemon(645)
│   └─multipathd(358)─┬─{multipathd}(370)
│                     │   └─{multipathd}(372)
│                     │   └─{multipathd}(373)
│                     │   └─{multipathd}(374)
│                     │   └─{multipathd}(375)
│                     └─{multipathd}(376)
│
│   └─polkitd(651)─┬─{gdbus}(754)
│                  │   └─{gmain}(751)
│                  └─{pool-spawner}(752)
│
│   └─rsyslogd(744)─┬─{in:imklog}(770)
│                   │   └─{in:imuxsock}(769)
│                   └─{rs:main Q:Reg}(771)
│
│   └─sshd(910)──sshd(911)──sshd(913)──bash(914)─┬─mainprocess(944)─┬─mainprocess(945)─┬─subprocess1(947)
│                                                │                               │   └─subprocess1(949)
│                                                │                               └─mainprocess(946)─┬─subprocess2(948)
│                                                │                               │   └─subprocess2(950)
│                                                │                               └─subprocess2(951)
│                                                └─pstree(1027)
│
│   └─systemd-journal(303)
│   └─systemd-logind(666)
│   └─systemd-network(519)
│   └─systemd-resolve(535)
│   └─systemd-timesyn(543)──{sd-resolve}(568)
│   └─systemd-udev(368)
│   └─udisksd(669)─┬─{cleanup}(797)
│                   │   └─{gdbus}(697)
│                   │   └─{gmain}(694)
│                   │   └─{pool-spawner}(695)
│                   └─{probing-thread}(765)
│
│   └─unattended-upgr(750)──{gmain}(826)
```

Question 8

Use `renice` command to change priority level of one of process “subprocess1”.

Command
<code>renice -n 10 -p 947</code>
Output
<pre>fathi@secr2043:~\$ ps -C subprocess1 PID TTY TIME CMD 947 pts/0 00:00:00 subprocess1 949 pts/0 00:00:00 subprocess1 fathi@secr2043:~\$ renice -n 10 -p 947 947 (process ID) old priority 0, new priority 10</pre>

Question 9

Terminate all running processes with the name “mainprocess”.

Command
<code>pkill mainprocess</code>
Output
<pre>fathi@secr2043:~\$ pkill mainprocess Main process (ID: 945) received signal: 15. Terminating... Main process (ID: 944) received signal: 15. Terminating... Main process (ID: 946) received signal: 15. Terminating... [1]+ Done ./mainprocess</pre>

Question 10

Write a short C or Python code (choose only one language) demonstrating multiprocessing with `fork()` and `wait()`. Compile and/or run the code. Show the output.

Source Code:

```
GNU nano 7.2                                fork_example.c *
#include <stdio.h>
#include <unistd.h>
#include <sys/wait.h>

int main() {
    pid_t pid = fork();

    if (pid < 0) {
        perror("fork failed");
        return 1;
    } else if (pid == 0) {
        printf("Child process: PID = %d\n", getpid());
    } else {
        wait(NULL);
        printf("Parent process: PID = %d, Child PID = %d\n", getpid(), pid);
    }

    return 0;
}
```

Output:

```
fathi@secr2043:~/fork_example$ nano fork_example.c
fathi@secr2043:~/fork_example$ gcc fork_example.c -o fork_example
fathi@secr2043:~/fork_example$ ./fork_example
Child process: PID = 2563
Parent process: PID = 2562, Child PID = 2563
```